

OPERATING SYSTEM

WEEK-03 PARTICAL

NAME :

P.Yaswanth Surya

ROLL

NO: 2520030595

SECTION :07

>Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
#include<unistd.h>
#include<stdio.h>
int main() {
int pid;
pid=fork();

if(pid<0)
printf("fork not successfully");
else if(pid==0){
printf("child process\n");
printf("the process id is %d\n",getpid());
printf("the child process id is %d\n",getppid());
}
else{
printf("parent process\n");
printf("the process id is %d\n",getpid());
```

```
printf("the parent proceidd id %d\n",getppid());  
}  
}
```

```
yaswanthp@Yaswanths-MacBook-Air-M1:~$ nano partical3.c  
yaswanthp@Yaswanths-MacBook-Air-M1:~$ gcc partical3.c  
yaswanthp@Yaswanths-MacBook-Air-M1:~$ ./a.out  
parent process  
the process id is 492  
the parent proceidd id 324  
child process  
the process id is 493  
the child process id is 323  
yaswanthp@Yaswanths-MacBook-Air-M1:~$
```

>Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations

The fork() system call creates a new child process from the parent process. The child process has a unique PID, while getppid() displays the PID of its parent process.

- When pid == 0, the child process executes and displays its PID and parent's PID.
- When pid > 0, the parent process executes and displays its own PID and its parent process's PID.

- Both parent and child execute independently after `fork()`.
- The output order of the parent and child may vary because process scheduling is controlled by the Linux operating system.
- If `fork()` fails, `pid < 0` and the program displays "fork not successfully".